GET et POST

Complément de la séance 1 – deux méthodes, deux intentions – version étudiant

Pour approfondir la partie « Traiter un formulaire » de la séance 1
Environnement : XAMPP, VSCodium

Présentation du complément

En séance 1, vous avez écrit des formulaires en `method="post"` et lu `$_GET` et `$_POST`.

Ce complément approfondit une question qui paraît simple et qui est source de nombreuses erreurs : **quand utiliser GET, quand utiliser POST, et pourquoi ?** Vous allez regarder ce que le navigateur envoie réellement, écrire une recherche en GET, un formulaire en POST sans doublon, puis constater qu'un lien en GET qui modifie des données est dangereux.

Durée indicative : 1 heure, en autonomie.

Compétences visées

- Décrire la structure d'une requête HTTP et dire où voyagent les données en GET et en POST.
- Distinguer une méthode **sûre** et **idempotente** (GET) d'une méthode qui modifie l'état du serveur (POST).
- Écrire une page GET robuste : valeurs par défaut, liste blanche, entiers bornés, liens de pagination.
- Écrire un formulaire POST qui n'enregistre pas deux fois la même demande (motif PRG).
- Expliquer pourquoi une action qui modifie des données ne doit jamais être déclenchable par GET.
- Corriger les idées reçues les plus courantes (« POST est sécurisé », « POST est illimité »…).

Prérequis : la séance 1 (formulaires, validation, `h()`, `champ()`, PRG).

Déroulé conseillé

Étape	Durée	Contenu
Parties 1 à 4	lecture	Le cours : HTTP, sémantique, comparaison, PHP
Exercice A	10 min	Observer une requête GET et une requête POST
Exercice B	15 min	Une recherche en GET (filtre, tri, pagination)
Exercice C	15 min	Un formulaire de contact en POST, sans doublon
Exercice D	10 min	Le lien qui vide le panier
Exercice E	bonus	`fetch` : GET avec paramètre et POST avec corps JSON

Partie 1 – Ce que dit HTTP

Anatomie d'une requête

Une requête HTTP comporte trois parties : une **ligne de requête** (méthode, adresse, version), des **en-têtes**, puis éventuellement un **corps**. Le navigateur fabrique la requête à partir de l'attribut `method` du formulaire.



Une requête GET et une requête POST telles qu'elles circulent sur le réseau

- En **GET**, les données du formulaire sont ajoutées à l'adresse, après un `?`, sous la forme `nom=valeur` séparés par `&` : la *chaîne de requête* (`query string`). La requête n'a pas de corps.
- En **POST**, la ligne de requête ne contient pas les données : elles sont placées dans le **corps**, et les en-têtes `Content-Type` et `Content-Length` en décrivent le format et la taille.

> Un point de vocabulaire : on ne parle pas de « paramètres d'URL » pour les données d'un POST. Ce sont des données du **corps** de la requête.

L'encodage des données

Une adresse ne peut contenir que certains caractères. Tous les autres sont écrits sous forme `%` suivi de deux chiffres hexadécimaux (*encodage pourcentage*), sur la base de leur codage UTF-8. Les formulaires utilisent la même règle pour le corps d'un POST.

Caractère	Encodé	Remarque
é	`%C3%A9`	Les accents (et les emojis) passent sans problème
espace	`+` (ou `%20`)	`+` dans les formulaires
&	`%26`	Sinon il séparerait deux paramètres
@	`%40`	

PHP décode tout cela automatiquement : `$_GET['message']` contient `café & thé`.

Le type du corps d'un POST dépend de l'attribut `enctype` du formulaire :

enctype	Usage	Où PHP le met
`application/x-www-form-urlencoded` (défaut)	Formulaires ordinaires	`$_POST`
`multipart/form-data`	Formulaires avec fichiers (`<input type="file">`)	`$_POST` et `$_FILES`
`application/json` (avec `fetch`)	Échanges avec une API	**Pas** dans `$_POST` : il faut lire `php://input`

La réponse et son code de statut

Le serveur répond avec un **code de statut** :

Code	Signification	Exemple d'usage dans nos TP
200	Succès	Page affichée
302, 303	Redirection (le navigateur demande une autre adresse **en GET**)	Après un POST réussi (motif PRG)
400	Requête mal formée	Identifiant invalide, JSON incorrect
403	Accès interdit	Mode organisateur absent, jeton CSRF faux
405	Méthode non autorisée	`GET` sur une page qui n'accepte que `POST`
415	Type de contenu non pris en charge	Corps non JSON envoyé à une API JSON

Partie 2 – Deux méthodes, deux intentions

Sûre et idempotente

HTTP donne aux méthodes un **sens**, qui n'est pas qu'une question de présentation :

- une méthode est **sûre** si elle ne modifie rien sur le serveur : elle sert à **lire** ;
- une méthode est **idempotente** si la répéter plusieurs fois a le même effet que la faire une seule fois.

Méthode	Intention	Modifie des données ?	Sûre	Idempotente
GET	Lire une ressource	Non	Oui	Oui
POST	Envoyer des données, créer, déclencher une action	Oui	Non	Non
PUT, PATCH, DELETE	Remplacer, modifier, supprimer	Oui	Non	PUT et DELETE : oui

Les formulaires HTML ne connaissent que GET et POST. Les autres méthodes servent surtout aux API, avec `fetch`.

Pourquoi c'est important

Parce que GET est supposé sans effet de bord, **tout le monde se permet de le rejouer** :

- le navigateur recharge la page sans avertissement ;
- il peut **précharger** les liens, et les moteurs de recherche et les robots **suivent tous les liens** qu'ils trouvent ;
- une adresse peut être mise en favori, envoyée par message ou ouverte dans une balise `<img>`.

Si un lien `panier.php?vider=1` vide le panier, il suffit d'un robot, d'un préchargement ou d'une page piégée pour que l'action soit déclenchée. À l'inverse, pour un POST, le navigateur demande confirmation avant de renvoyer le formulaire, justement parce que l'action peut être rejouée (doublon de commande, par exemple).

> **Règle :** une action qui modifie des données (création, modification, suppression, déconnexion, achat) **n'est jamais déclenchée par GET**.

Partie 3 – Comparaison complète

	GET	POST
Où voyagent les données	Dans l'URL (chaîne de requête)	Dans le corps de la requête
Visible dans la barre d'adresse	Oui	Non (mais visible dans les outils de développement)
Favori, historique, journaux d'accès du serveur, en-tête `Referer`	Les paramètres sont conservés avec l'URL	Seule l'URL est conservée, sans les données
Mise en cache	Possible	En général non
Rechargement de la page / bouton « Précédent »	Rejoué sans avertissement	Le navigateur demande confirmation du renvoi
Taille des données	Pas de limite dans HTTP, mais des limites pratiques (voir ci-dessous)	Limite fixée par la configuration du serveur
Type des données	Texte, encodé en pourcentage	Texte ou binaire (`multipart/form-data`)
Effet sur le serveur	Aucun (méthode sûre)	Peut modifier l'état
Cas d'usage	Recherche, filtres, tri, pagination	Inscription, connexion, création, envoi de fichier

À propos des limites. Le protocole HTTP ne fixe pas de longueur maximale d'URL, mais navigateurs, serveurs et intermédiaires en imposent une : on considère qu'une URL jusqu'à environ 2 000 caractères passe partout, et au-delà elle peut être refusée (par exemple, Apache refuse par défaut les lignes de requête de plus de 8 190 octets, avec l'erreur 414). Pour un POST, la limite dépend de la configuration : en PHP, `post_max_size` (souvent 8 Mo) et `max_input_vars` (1 000 champs) ; pour les fichiers, `upload_max_filesize`.

Idées reçues

Idée reçue	Réalité
« POST est sécurisé, GET ne l'est pas. »	**Faux.** Ni l'un ni l'autre ne chiffre quoi que ce soit : le corps d'un POST est lisible par qui observe la requête. POST évite seulement d'exposer les données dans l'URL, l'historique, les favoris et les journaux. Pour protéger le contenu, il faut **HTTPS**.
« Les données du POST sont invisibles. »	Invisibles dans la barre d'adresse, pas dans les outils de développement, ni pour un attaquant qui forge sa propre requête. **Toute donnée reçue est non fiable** et doit être validée, quelle que soit la méthode.
« Une action en POST est protégée. »	Non : un POST peut être déclenché depuis une autre page (**CSRF**, voir le complément JSON et CSRF). Il faut en plus un jeton.
« POST évite les doublons. »	Non : c'est justement le **rechargement d'un POST** qui crée des doublons. La parade est la redirection après POST (motif PRG).
« GET ne peut transporter que de l'ASCII. »	L'URL n'accepte que des caractères « sûrs », mais l'encodage pourcentage permet d'y mettre n'importe quel texte (accents, emojis). C'est le **binaire** (fichiers) qui n'a pas sa place dans une URL.
« POST est illimité. »	Non : il est limité par la configuration du serveur (`post_max_size`).
« Un formulaire d'inscription peut être en GET. »	Non : il envoie des données personnelles (ici dans l'URL, donc dans l'historique et les journaux) et crée un compte, ce qui modifie des données.

Partie 4 – GET et POST en PHP

Lire les données

Besoin	Code
Savoir quelle méthode a été utilisée	`$_SERVER['REQUEST_METHOD']` (`'GET'` ou `'POST'`)
Lire un paramètre d'URL	`$_GET['q'] ?? ''`
Lire un champ de formulaire POST	`$_POST['email'] ?? ''`
Lire un corps JSON	`file_get_contents('php://input')` puis `json_decode`
Lire un fichier envoyé	`$_FILES['photo']`

`$_REQUEST` mélange `$_GET`, `$_POST` et les cookies : on **l'évite**, car on ne sait plus d'où vient une donnée.

Comme en séance 1, on lit avec `??` et on vérifie le type, car un client peut envoyer `?q[]=x` (un tableau) :

```php
<?php
declare(strict_types=1);

function h(string $texte): string
{
    return htmlspecialchars($texte, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}

/** Lit un champ de formulaire POST (texte ou chaîne vide). */
function champ(string $cle): string
{
    $valeur = $_POST[$cle] ?? '';
    return is_string($valeur) ? trim($valeur) : '';
}

/** Lit un paramètre d'URL GET (texte ou chaîne vide). */
function parametre(string $cle): string
{
    $valeur = $_GET[$cle] ?? '';
    return is_string($valeur) ? trim($valeur) : '';
}

function afficherErreur(array $erreurs, string $cle): void
{
    if (isset($erreurs[$cle])) {
        echo '<div class="erreur">' . h($erreurs[$cle]) . '</div>';
    }
}
```

Traiter une page en GET

Une recherche, un filtre ou une page de résultats ne doit **jamais planter**, quelle que soit l'adresse tapée. Chaque paramètre est donc remplacé par une valeur par défaut s'il est absent ou invalide :

```
$q     = mb_substr(parametre('q'), 0, 50);                  // texte, longueur bornée
$tri   = Tri::tryFrom(parametre('tri')) ?? Tri::Nom;        // liste blanche (enum)
$page  = filter_var(parametre('page'), FILTER_VALIDATE_INT, ['options' => ['min_range'
=> 1]]);
$page  = $page === false ? 1 : $page;                       // entier borné
```

Pour fabriquer un lien qui conserve les filtres, on construit la chaîne de requête avec `http_build_query` (qui encode correctement) puis on échappe le résultat avec `h()` pour l'insérer dans le HTML :

```
function urlRecherche(array $params): string
{
    return 'recherche.php?' . http_build_query($params);
}
// <a href="<?= h(urlRecherche(['q' => 'é', 'page' => 2])) ?>">Suivant</a>
// donne : href="recherche.php?q=%C3%A9&amp;page=2"
```

Traiter un formulaire en POST, puis rediriger

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1. lire  2. valider  3. traiter
    if ($erreurs === []) {
        $_SESSION['confirmation'] = 'Merci, message reçu.';
        header('Location: contact.php', true, 303);   // 303 See Other : le navigateur
recharge en GET
        exit;
    }
}
```

Après un POST réussi, on **redirige** vers une page en GET (motif **PRG**, *Post / Redirect / Get*). Le rechargement de la page par l'utilisateur rejoue alors le GET inoffensif au lieu de renvoyer le formulaire. Le code 303 (*See Other*) est fait pour cet usage ; le 302 que PHP utilise par défaut se comporte de la même façon dans les navigateurs actuels.

Choisir la méthode

Question	Si la réponse est oui
L'action **modifie** quelque chose (création, suppression, achat, déconnexion…) ?	**POST**
Les données sont personnelles ou sensibles (mot de passe, e-mail, message) ?	**POST** (et HTTPS)
Il y a un **fichier** à envoyer ?	**POST** avec `enctype="multipart/form-data"`
Le résultat doit-il pouvoir être mis en **favori**, partagé par lien, retrouvé dans l'historique (recherche, filtre, tri, page) ?	**GET**
Est-ce une simple **lecture**, sans effet de bord ?	**GET**

La version courte pour retenir : **GET pour lire, POST pour agir.**

TP complémentaire

Préparation (5 min)

Dans le dossier `tp-php` de la séance 1, ajouter à `fonctions.php` la fonction `parametre()` donnée dans la partie 4 (en plus de `h()` et `champ()`). Le serveur intégré (`php -S localhost:8000`) doit tourner.

Exercice A – Observer une requête (10 min)

Objectif : voir ce qui part du navigateur et ce que PHP en fait.

- Créer `requete.php`, qui affiche la méthode, l'adresse, la chaîne de requête, le type et la taille du corps, `$_GET`, `$_POST` et le corps brut :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';

$corps = file_get_contents('php://input');

$infos = [
    'Méthode ($_SERVER[REQUEST_METHOD])'  => $_SERVER['REQUEST_METHOD'],
    'Adresse demandée (REQUEST_URI)'       => $_SERVER['REQUEST_URI'],
    'Partie après le ? (QUERY_STRING)'     => $_SERVER['QUERY_STRING'] ?? '',
    'Type du corps (CONTENT_TYPE)'         => $_SERVER['CONTENT_TYPE'] ?? '(aucun)',
    'Taille du corps (CONTENT_LENGTH)'     => $_SERVER['CONTENT_LENGTH'] ?? '0',
    'Limite post_max_size de PHP'          => (string) ini_get('post_max_size'),
];
?>
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Ce que reçoit le serveur</title>
  <style>pre { background: #f3f4f8; padding: .6rem; } fieldset { margin-bottom: 1rem;
}</style>
</head>
<body>
  <h1>Ce que reçoit le serveur</h1>

  <fieldset>
    <legend>Formulaire GET</legend>
    <form method="get" action="requete.php">
      <input type="text" name="prenom" value="Zoé">
      <input type="text" name="message" value="café & thé">
      <button type="submit">Envoyer en GET</button>
    </form>
  </fieldset>

  <fieldset>
    <legend>Formulaire POST (application/x-www-form-urlencoded, par défaut)</legend>
    <form method="post" action="requete.php">
      <input type="text" name="prenom" value="Zoé">
      <input type="text" name="message" value="café & thé">
      <button type="submit">Envoyer en POST</button>
    </form>
  </fieldset>
```

```
  <fieldset>
    <legend>Formulaire POST (multipart/form-data)</legend>
    <form method="post" action="requete.php" enctype="multipart/form-data">
      <input type="text" name="prenom" value="Zoé">
      <input type="text" name="message" value="café & thé">
      <button type="submit">Envoyer en POST multipart</button>
    </form>
  </fieldset>

  <h2>Requête reçue</h2>
  <table border="1" cellpadding="6">
    <?php foreach ($infos as $nom => $valeur): ?>
      <tr><th align="left"><?= h($nom) ?></th><td><code><?= h((string) $valeur)
?></code></td></tr>
    <?php endforeach; ?>
  </table>

  <h3>$_GET</h3>
  <pre><?= h(var_export($_GET, true)) ?></pre>

  <h3>$_POST</h3>
  <pre><?= h(var_export($_POST, true)) ?></pre>

  <h3>Corps brut de la requête (php://input)</h3>
  <pre><?= h($corps === '' || $corps === false ? '(vide)' : $corps) ?></pre>
</body>
</html>
```

- Envoyer successivement les **trois formulaires** de la page. À chaque envoi, relever les informations affichées.
- Ouvrir les outils de développement (`F12`), onglet **Réseau**, puis refaire les envois. Pour chaque requête, observer la méthode, l'adresse, les en-têtes de requête (`Content-Type`) et le corps (onglet **Requête** ou **Payload**).
- *(Facultatif)* Depuis un second terminal, envoyer la même chose avec `curl` : `curl "http://localhost:8000/requete.php?prenom=Zoe"` puis `curl -d "prenom=Zoe" http://localhost:8000/requete.php` (sous Windows PowerShell, utiliser `curl.exe`).

Compléter le tableau :

Formulaire	`QUERY_STRING`	`CONTENT_TYPE`	`$_GET`	`$_POST`	Corps brut (`php://input`)
GET	?	?	?	?	?
POST ordinaire	?	?	?	?	?
POST multipart	?	?	?	?	?

À vérifier

- ☐ En GET, les données sont dans la barre d'adresse, encodées (`caf%C3%A9+%26+th%C3%A9`), et le corps est vide.
- ☐ En POST ordinaire, l'adresse ne contient aucune donnée ; le corps contient `prenom=Zo%C3%A9&message=...` et `$_POST` est rempli.
- ☐ En POST multipart, `$_POST` est rempli mais le corps brut est **vide** dans `php://input` : pourquoi ? (Indice : PHP a déjà lu et décomposé ce corps.)

- ☐ Les données du POST sont **visibles** dans l'onglet Réseau : que cela change-t-il à l'idée que « POST cache les données » ?

Exercice B – Une recherche en GET (15 min)

Objectif : une page de résultats partageable, robuste face à toute adresse.

Créer `recherche.php`, qui affiche les modules d'une école. Données et enum à reprendre :

```
enum Tri: string
{
    case Nom   = 'nom';
    case Annee = 'annee';
}

const PAR_PAGE = 3;

const MODULES = [
    ['nom' => 'Algorithmique',                'annee' => 1],
    ['nom' => 'Architecture des ordinateurs', 'annee' => 1],
    ['nom' => 'Bases de données',             'annee' => 2],
    ['nom' => 'Développement web',            'annee' => 1],
    ['nom' => 'Génie logiciel',               'annee' => 2],
    ['nom' => 'Intelligence artificielle',    'annee' => 3],
    ['nom' => 'Mathématiques discrètes',      'annee' => 1],
    ['nom' => 'Programmation C',              'annee' => 1],
    ['nom' => 'Réseaux',                      'annee' => 2],
    ['nom' => 'Systèmes d\'exploitation',     'annee' => 2],
];
```

Cahier des charges :

- Un formulaire en **GET** : champ texte `q`, liste `annee` (toutes, 1, 2, 3), liste `tri` (nom ou année).
- Les paramètres `q`, `annee`, `tri`, `page` sont lus avec `parametre()` ; chacun a une **valeur par défaut** si absent ou invalide (`q` limité à 50 caractères, `tri` vérifié avec `Tri::tryFrom`, `annee` entre 1 et 3, `page` entier ≥ 1 ramené à la dernière page si trop grand).
- Les résultats sont filtrés (`mb_stripos` pour la recherche, insensible à la casse), triés, puis paginés par 3.
- Les liens « Précédent » et « Suivant » **conservent les filtres** (`http_build_query`, puis `h()`).
- Les formulaires et valeurs réaffichées sont échappés avec `h()`.

À vérifier

- ☐ Copier l'adresse d'une page de résultats dans un nouvel onglet donne **le même résultat** (c'est l'intérêt du GET).
- ☐ `recherche.php?page=999` affiche la dernière page ; `?page=abc&annee=9&tri=n'importe-quoi` affiche une page valide, sans erreur.
- ☐ `recherche.php?q[]=x` (un tableau) ne provoque ni erreur ni avertissement.
- ☐ La saisie `<script>alert(1)</script>` dans `q` s'affiche en texte, sans s'exécuter.
- ☐ Les liens de pagination conservent le filtre et encodent correctement les caractères spéciaux (`q=é` donne `q=%C3%A9`).
- ☐ Question : pourquoi une recherche est-elle un bon candidat pour GET, alors qu'un formulaire d'inscription ne l'est pas ?

Exercice C – Un formulaire de contact sans doublon (15 min)

Objectif : constater le problème du rechargement d'un POST, puis le corriger avec le motif PRG.

Version 1 (sans redirection). Créer `contact_v1.php` : formulaire POST avec `email` (valide) et `message` (5 à 500 caractères). Quand tout est valide, **sans rediriger**, incrémenter un compteur en session (`$_SESSION['recus']`), afficher « Merci, votre message a bien été reçu » et le nombre de messages reçus pendant la session.

1. Envoyer un message valide : le compteur vaut 1.
2. Appuyer sur **F5** : le navigateur demande de confirmer le renvoi du formulaire. Confirmer. Que vaut le compteur ?

Version 2 (avec PRG). Copier le fichier en `contact.php`. Après un POST réussi, stocker le message de confirmation dans `$_SESSION['confirmation']`, **rediriger** en 303 vers `contact.php` puis `exit`. Au chargement de la page, lire le message et le supprimer de la session.

3. Refaire le test : envoyer un message valide, puis appuyer sur F5.

À vérifier

- ☐ Version 1 : après F5 et confirmation, le compteur passe à 2 : le même message a été « reçu » deux fois.
- ☐ Version 2 : aucun avertissement de renvoi au rechargement ; le compteur reste à 1.
- ☐ Le message de confirmation ne s'affiche qu'une fois.
- ☐ Un e-mail invalide ou un message trop court affiche les erreurs sans redirection (le formulaire garde les valeurs saisies).
- ☐ Question : pourquoi le compteur est-il resté à 1 en version 2, alors que la page s'est rechargée ? Quelle méthode est utilisée par le rechargement ?

Exercice D – Le lien qui vide le panier (10 min)

Objectif : comprendre le danger d'une action déclenchée par GET.

`panier_v1.php` affiche un panier stocké en session, avec un lien « Vider le panier » (en GET) et un bouton « Remplir à nouveau » (en POST). Copier ce fichier :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';

session_start();

if (!isset($_SESSION['panier'])) {
    $_SESSION['panier'] = ['Clavier', 'Souris', 'Écran'];
}

// DANGEREUX : une simple requête GET modifie les données
if (parametre('vider') === '1') {
    $_SESSION['panier'] = [];
    header('Location: panier_v1.php');
```

```
    exit;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST' && champ('action') === 'remplir') {
    $_SESSION['panier'] = ['Clavier', 'Souris', 'Écran'];
    header('Location: panier_v1.php', true, 303);
    exit;
}

$panier = $_SESSION['panier'];
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Panier</title></head>
<body>
  <h1>Mon panier (<?= count($panier) ?>)</h1>
  <ul>
    <?php foreach ($panier as $article): ?>
      <li><?= h($article) ?></li>
    <?php endforeach; ?>
  </ul>

  <p><a href="panier_v1.php?vider=1">Vider le panier</a></p>

  <form method="post" action="panier_v1.php">
    <input type="hidden" name="action" value="remplir">
    <button type="submit">Remplir à nouveau</button>
  </form>
</body>
</html>
```

Créer ensuite une page « piégée », `piege.html`, dans le même dossier :

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Photos de chatons</title></head>
<body>
  <h1>Les plus beaux chatons du web</h1>
  <p>Chargement des photos…</p>

  <!-- Une « image » dont l'adresse est une action : le navigateur la demande en GET,
cookies compris. -->
  <img src="panier_v1.php?vider=1" width="1" height="1" alt="">
</body>
</html>
```

1. Ouvrir `panier_v1.php` : le panier contient trois articles.
2. Dans le même navigateur, ouvrir `http://localhost:8000/piege.html`.
3. Revenir sur `panier_v1.php`.
4. **Corriger :** copier le fichier en `panier.php` et faire de « Vider le panier » un **bouton de formulaire POST** (par exemple `name="action" value="vider"`) ; ignorer toute requête GET qui tenterait de modifier le panier ; après un POST, rediriger en 303 vers `panier.php`. Remplacer l'adresse de l'image de `piege.html` par `panier.php?vider=1` et recommencer.

À vérifier

- ☐ Avec `panier_v1.php`, le panier est vide **sans qu'aucun clic ne soit fait** sur le lien : la balise `<img>` a demandé l'adresse en GET, cookies compris.
- ☐ Avec `panier.php`, la page piégée est sans effet (le GET ne modifie rien) ; le bouton POST fonctionne.
- ☐ Question : un robot d'indexation qui parcourt votre site suivrait-il le lien `panier_v1.php?vider=1` ? Et un bouton POST ?
- ☐ Question : le POST suffit-il à protéger l'action contre une page piégée qui enverrait un **formulaire** caché ? (Indice : complément « JSON et jeton CSRF ».)

Exercice E (bonus) – GET et POST avec `fetch` (15 min)

Objectif : voir ce qui change quand le corps est du JSON.

`api_echo.php` répond selon la méthode reçue :

```
<?php
declare(strict_types=1);

header('Content-Type: application/json; charset=utf-8');

$methode = $_SERVER['REQUEST_METHOD'];

if ($methode === 'GET') {
    $q = $_GET['q'] ?? '';
    echo json_encode(
        ['methode' => 'GET', 'q' => is_string($q) ? $q : ''],
        JSON_UNESCAPED_UNICODE
    );
    exit;
}

if ($methode === 'POST') {
    $type = $_SERVER['CONTENT_TYPE'] ?? '';
    if (!str_starts_with($type, 'application/json')) {
        http_response_code(415);                    // 415 : type de contenu non pris
en charge
        echo json_encode(['erreur' => 'Un corps JSON est attendu.']);
        exit;
    }

    try {
        $donnees = json_decode((string) file_get_contents('php://input'), true, 512,
JSON_THROW_ON_ERROR);
    } catch (JsonException) {
        http_response_code(400);                    // 400 : requête mal formée
        echo json_encode(['erreur' => 'JSON invalide.']);
        exit;
    }

    echo json_encode(
        ['methode' => 'POST', 'recu' => $donnees, 'post_superglobale' => $_POST],
        JSON_UNESCAPED_UNICODE
    );
    exit;
}

http_response_code(405);                            // 405 : méthode non autorisée
```

```
header('Allow: GET, POST');
echo json_encode(['erreur' => 'Méthode non autorisée.']);
```

`echo.html` envoie une requête GET puis une requête POST JSON avec `fetch` :

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>GET et POST avec fetch</title></head>
<body>
  <h1>GET et POST avec fetch</h1>
  <button id="get">GET avec paramètre</button>
  <button id="post">POST avec corps JSON</button>
  <pre id="resultat"></pre>

  <script>
    const texte = 'café & thé';
    const sortie = document.getElementById('resultat');

    async function afficher(promesse) {
      const reponse = await promesse;
      sortie.textContent = `HTTP ${reponse.status}\n` + JSON.stringify(await
reponse.json(), null, 2);
    }

    document.getElementById('get').addEventListener('click', () => {
      const adresse = 'api_echo.php?' + new URLSearchParams({ q: texte });   // encode
le « & » et les accents
      afficher(fetch(adresse));
    });

    document.getElementById('post').addEventListener('click', () => {
      afficher(fetch('api_echo.php', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ q: texte }),
      }));
    });
  </script>
</body>
</html>
```

- Ouvrir `http://localhost:8000/echo.html` et cliquer sur chaque bouton. Observer la réponse et les requêtes dans l'onglet Réseau.
- Dans un terminal, envoyer un POST de formulaire ordinaire à `api_echo.php` : `curl -d "q=abc" http://localhost:8000/api_echo.php`.
- Envoyer un JSON invalide : `curl -H "Content-Type: application/json" -d "{\"q\":" http://localhost:8000/api_echo.php`.
- Envoyer une méthode non prévue : `curl -X PUT http://localhost:8000/api_echo.php`.

À vérifier

- ☐ La requête GET contient `q=caf%C3%A9+%26+th%C3%A9` dans l'adresse : `URLSearchParams` a encodé l'espace, l'accent et le `&`.
- ☐ Pour le POST JSON, `post_superglobale` est **vide** (`[]`) alors que `recu` contient la donnée : pourquoi `$_POST` est-il vide ?

- ☐ Les trois erreurs renvoient respectivement les codes **415**, **400** et **405** (avec l'en-tête `Allow: GET, POST`).
- ☐ Question : à quoi sert le code de statut, alors que le corps de la réponse contient déjà un message d'erreur ?

Annexes

A. Aide-mémoire

Besoin	Code
Méthode de la requête	`$_SERVER['REQUEST_METHOD']`
Paramètre d'URL / champ de formulaire	`parametre('q')` / `champ('email')`
Construire une chaîne de requête	`http_build_query(['q' => 'é', 'page' => 2])`
Encoder / décoder une valeur isolée	`urlencode($v)` / `urldecode($v)`
Lire un entier borné	`filter_var($v, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]])`
Redirection après POST	`header('Location: page.php', true, 303); exit;`
Refuser une méthode	`http_response_code(405); header('Allow: POST'); exit;`
Corps brut / JSON	`file_get_contents('php://input')` / `json_decode(..., true, 512, JSON_THROW_ON_ERROR)`
Type du corps	`$_SERVER['CONTENT_TYPE'] ?? ''`
Limites PHP	`ini_get('post_max_size')`, `ini_get('max_input_vars')`
Formulaire avec fichier	`<form method="post" enctype="multipart/form-data">` et `$_FILES`

B. Erreurs fréquentes et diagnostic

Symptôme	Cause probable	Solution
`$_POST` est vide	Formulaire en `method="get"`, ou corps JSON, ou corps trop gros (`post_max_size`)	Vérifier la méthode et le `Content-Type` dans l'onglet Réseau ; lire `php://input` pour du JSON
`$_GET` est vide alors que l'adresse a des paramètres	Le formulaire est en POST, ou les paramètres sont après un # (jamais envoyé au serveur)	Vérifier `method` ; regarder `$_SERVER['QUERY_STRING']`
`Undefined array key "q"`	Lecture directe de `$_GET['q']`	`parametre('q')` ou `?? ''`
`Array to string conversion`	Un client a envoyé `?q[]=x`	Tester `is_string()` (c'est le rôle de `parametre()` et `champ()`)
La pagination perd le filtre	Lien construit sans `$communs`	Reconstruire l'adresse avec `http_build_query`
`&` visible dans la barre d'adresse	Lien échappé deux fois	Un seul `h()`, au moment d'écrire l'attribut `href`
Message « Confirmer le renvoi du formulaire »	Page affichée directement par un POST	Ajouter la redirection (motif PRG)
Doublon après un rechargement	Idem	Idem
`405 Method Not Allowed` sur une page qui devrait répondre	Mauvaise méthode (`GET` sur une page `POST` seulement)	Vérifier le formulaire ou la requête `fetch`
`Cannot modify header information – headers already sent`	Sortie avant `header()`	Placer la logique avant tout HTML (voir séance 1)
La page piégée n'a aucun effet (exercice D)	Mauvais navigateur ou mauvaise session	Utiliser le **même** navigateur et vérifier que le panier est plein

C. Pour aller plus loin

- **Protection des actions POST :** jeton CSRF (complément « JSON et jeton CSRF »).
- **Autres méthodes et API REST :** PUT, PATCH, DELETE ; codes de statut ; formats d'échange en JSON.
- **Cache HTTP :** en-têtes `Cache-Control`, `ETag` ; pourquoi un GET peut être mis en cache et pas un POST.
- **Transport :** HTTPS, qui chiffre l'URL, les en-têtes et le corps pendant le trajet ; l'adresse (nom de domaine) reste en revanche visible de qui observe le réseau.
- **Références :** le manuel PHP (`php.net/manual/fr/reserved.variables.get.php`, `...post.php`) et la documentation des méthodes HTTP de MDN.